DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIVIDUAL CONTRIBUTION REPORT

Team Assignment

Conflict-Free Exam Timetable Scheduling Using Backtracking (Graph Colouring)

Department	Computer Science & Engineering
Programme	B.E/B.Tech Computer Science & Engineering
Course Code & Course Name	CSA0614 – Design and Analysis of Algorithms
Academic Year / Batch	2026-27
Faculty Name	Dr. A. Sairam
Assignment Title	Conflict-Free Exam Timetable Scheduling Using Backtracking (Graph Colouring)
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4, CO7
Bloom's Taxonomy Level	L4 – L5 (Analyzing, Evaluating)
SDG Mapping	SDG 4 – Quality Education
Industry / Societal Relevance	Universities and colleges must build clash-free exam timetables every semester; this assignment mirrors that real scheduling problem using the Graph Colouring formulation taught in the Backtracking unit.

Team Members

S.No	Team Member	Register No.
1	K. Jaya Venkata Sai	192525182
2	Athavan K.	192524036
3	G. Rama Krishna	192565028
4	Manoj Kumar K.	192521041

Date of Submission: 02 – 09 – 2026

Department of Computer Science and Engineering

Individual Contribution Report — Team Assignment

To be completed by EACH student individually and submitted along with the team's assignment. This report explains what YOU specifically did, so it must be written in your own words and reflect only your own work.

A. Assignment & Team Information

Course Code & Name	CSA0614 – Design and Analysis of Algorithms
Assignment Title	Conflict-Free Exam Timetable Scheduling Using Backtracking (Graph Colouring)
Team / Group No.	Team 5
Team Members (all names)	K. Jaya Venkata Sai – 192525182 Athavan K. – 192524036 G. Rama Krishna – 192565028 Manoj Kumar K. – 192521041
Date of Submission	02-09-2026

B. Student's Own Details

Student Name	Athavan k
Register / Roll No.	192524036
Role in the Team	Complexity Analysis

C. Task-wise Individual Contribution

List every task/module you personally worked on. Add rows if needed. Do not describe work done by teammates.

S.No	Task / Module Handled	Description of Work Done (in your own words)	Tools / Techniques Used	Approx. Hours Spent

1	Implementation, Complexity & Testing	Implemented the Plain Backtracking and Pruning-Enhanced Backtracking algorithms in Python, generated randomized conflict graphs with at least 15 courses, analysed $O(k^n)$ worst-case complexity, and tested runtime, assignments and backtracks.	Python, Backtracking, Graph Colouring, MCV, Forward Checking, Performance Testing	1 hour
2	Complexity Analysis	Derived and documented the worst-case time complexity $O(k^n)$ and analysed the practical effect of pruning on the search space.	Asymptotic Analysis, $O(k^n)$, Backtracking Analysis	1 hour
3	Testing & Performance Evaluation	Tested both algorithms and recorded assignments, backtracks and runtime to compare plain Backtracking with the pruning-enhanced approach.	Python, Runtime Measurement, Test Cases, Performance Metrics	1 hour
4	Results & Validation	Validated the generated colourings against conflict constraints and organized experimental results for comparison and reporting.	Graph Validation, Experimental Results, Data Analysis	1 hour

D. Team Contribution Percentage Summary

To be filled identically by every team member so contributions are consistent and sum to 100%. Disagreements should be discussed and resolved before submission.

Team Member Name	Register No.	% Contribution	Signature
K. Jaya Venkata Sai	192525182	25%	K. Jaya Venkata Sai
Athavan K.	192524036	25%	Athavan K.
G. Rama Krishna	192565028	25%	G. Rama Krishna
Manoj Kumar K.	192521041	25%	Manoj Kumar K.
	Total	100%	

E. Course Outcomes / Skills Applied

Course Outcome(s) Addressed by MY Work	How I Applied It
CO2, CO3, CO4, CO7	Applied algorithmic analysis, implementation, testing, performance evaluation and reflection to the graph-colouring based timetable scheduling problem.
L4 – L5	Analysed worst-case complexity, compared search behaviour and evaluated the effect of pruning.
SDG 4 – Quality Education	The project addresses efficient and reliable academic examination scheduling.

F. Challenges Faced and How I Resolved Them

Describe any technical or teamwork challenges you personally faced, and the steps you took to resolve them.

1. I initially found it challenging to compare the performance of Plain Backtracking and Pruning-Enhanced Backtracking because both algorithms could produce valid colourings. I resolved this by recording assignments, backtracks and runtime for the same input graphs.
2. Generating and testing larger conflict graphs was another challenge. I resolved it by using a randomized graph generator with fixed seeds so that the experiments were reproducible.

G. Individual Learning Outcome / Reflection

What did YOU personally learn from completing your part of this assignment? How did it build on classroom concepts?

1. I learned how the theoretical Backtracking and Graph Colouring concepts taught in class can be converted into a working Python implementation and evaluated using practical metrics.
2. I learned that heuristics such as Most-Constrained-Vertex ordering and Forward Checking can reduce the effective search space, which helped me understand the practical importance of algorithm optimization.

H. Declaration

I declare that the contribution described above accurately represents the work I personally performed for this team assignment, and that I have not misrepresented the work of any other team member as my own.

Student Signature	Date
Athavan K	02-09-2026

ABSTRACT

University examination scheduling can be represented as a graph-colouring problem. Each course is a vertex, an edge joins two courses that share at least one student, and an available examination period is represented by a colour. A correct timetable is therefore a proper colouring: adjacent vertices must have different colours.

This report develops and evaluates Module 2, the plain backtracking solution. Vertices are considered in the fixed order C1, C2, …, C6 and each vertex is tested against colours 1, 2, …, k. When a colour conflicts with an already coloured neighbour, it is rejected. When a later vertex cannot be assigned any colour, the algorithm undoes the most recent assignment and explores the next alternative.

For the Annexure A graph, the algorithm finds a valid colouring with three colours: C1 = 1, C2 = 2, C3 = 3, C4 = 1, C5 = 2, and C6 = 3. The k = 4 run also succeeds immediately because the first legal colour is always selected. The graph is not 2-colourable, and the additional k = 2 trace demonstrates genuine backtracking and failure. A deterministic experiment on random graphs with 15 to 40 vertices shows how the number of recursive calls and colour attempts can increase sharply even though the implementation is short and exact.

The central finding is that plain backtracking is complete and easy to verify, but its worst-case search space is exponential, $O(k^n)$. It is appropriate for small academic instances and as a reliable baseline for comparing pruning-enhanced methods; for larger dense conflict graphs, heuristics or constraint-programming techniques become important.

TABLE OF CONTENTS

LIST OF FIGURES AND TABLES

Figures

Tables

1. EXECUTIVE SUMMARY AND REPORT NAVIGATION

The report is deliberately focused on Module 2 rather than presenting all four proposed modules as if they had been completed. The target is the baseline algorithm required in Part A: fixed-order, colour-by-colour backtracking with trace logging.

The reader can follow the work in five layers. First, the scheduling story is converted into a graph-colouring model. Second, the recursive algorithm is specified independently of programming language. Third, the exact Annexure A instance is hand-traced for the required values k = 3 and k = 4. Fourth, the implementation is described and tested. Finally, measured search behaviour is connected to $O(k^n)$, the practical limits of exhaustive search, and the motivation for later pruning.

Evidence layer	Where it appears	What it demonstrates
Model	Sections 3 and 7	The input graph represents student conflicts.
Algorithm	Sections 8–10	The recursive search is unambiguous and implementable.
Trace	Sections 12–14 and Appendix B	Assignments, rejections, and undo operations are visible.
Validation	Sections 17–18	Correctness and scalability are checked on multiple inputs.
Analysis	Section 19	The final decision is justified by complexity and evidence.

The final recommended interpretation is simple: plain backtracking is the correct baseline because it is complete, deterministic, and easy to audit. It should not be mistaken for a scalable production scheduler without additional ordering heuristics, forward checking, or a more specialized constraint solver.

2. INTRODUCTION AND CONTEXT

A university examination timetable must coordinate courses, students, rooms, invigilators, and available periods. The present assignment isolates one important constraint: two courses sharing a student must not be scheduled in the same time slot. This abstraction is valuable because it captures a real operational conflict while keeping the algorithmic problem precise.

Graph colouring is a natural representation. A vertex denotes a course. A pair of vertices is adjacent when the corresponding courses have at least one student in

common. A colour denotes an examination period. A proper vertex colouring ensures that every conflicting pair receives different periods.

The central computational difficulty is that a locally legal choice can create a dead end later. The algorithm must therefore be able to retract a choice and try another. This is the defining role of backtracking: it performs a depth-first exploration of candidate partial solutions while abandoning a branch as soon as it violates the constraints.

Why Module 2 matters

- It provides a transparent baseline against which pruning-enhanced algorithms can be compared.
- It exposes the relationship between recursion, state restoration, and exhaustive search.
- It produces a trace that can be checked by hand, not only a final colour array.
- It reveals why a correct algorithm can still become impractical as n and k grow.

3. PROBLEM STATEMENT AND FORMAL FORMULATION

3.1 Problem statement

Given a course-conflict graph G = (V, E) and a fixed number of available examination slots k, assign a colour to every vertex such that no edge has equal-coloured endpoints. If such an assignment exists, return the timetable; otherwise, report that k slots are insufficient for the supplied graph.

3.2 Mathematical formulation

Let V = {C1, C2, …, Cn}. Let x_i be the colour assigned to vertex C_i, where $x_i \in \{1, 2, \ldots, k\}$. For every conflict edge $(Ci, Cj) \in E$, the constraint is $x_i \neq x_j$. The decision problem is:

Find $x = (x_1, x_2, \ldots, x_n)$ such that $x_i \in \{1, \ldots, k\}$ and $x_i \neq x_j$ for every $(Ci, Cj) \in E$.

The optimization interpretation is to determine the smallest k for which a valid assignment exists. Module 2 accepts k as an input and solves the decision version. It does not use a heuristic to choose the next vertex; the order is fixed and known in advance.

Symbol	Meaning	Value in Annexure A
G = (V, E)	Conflict graph	6 vertices, 8 edges
V	Set of course vertices	{C1, C2, C3, C4, C5, C6}
E	Pairs sharing at least one student	Eight listed pairs

Symbol	Meaning	Value in Annexure A
k	Available time slots / colours	3 and 4 required; 2 diagnostic
x_i	Colour of vertex i	Integer from 1 to k

4. OBJECTIVES AND EXPECTED OUTCOMES

4.1 Objectives

1. Model the examination scheduling problem as a graph-colouring decision problem.
2. Implement fixed-order plain backtracking for an arbitrary graph and a fixed k.
3. Show every colour attempt, conflict rejection, assignment, undo, and final outcome.
4. Hand-trace the Annexure A graph for k = 3 and k = 4.
5. Demonstrate an infeasible case with k = 2 so the purpose of backtracking is visible.
6. Validate the implementation with positive, negative, boundary, and randomized tests.
7. Relate measured search effort to the worst-case $O(k^n)$ time bound.

4.2 Expected outcomes

- A reproducible Python implementation with no hidden ordering heuristic.
- A valid 3-colour timetable for the Annexure A graph.
- A valid 4-colour timetable for the same graph.
- A clear failure report for 2 colours.
- Trace metrics: recursive calls, colour attempts, backtrack events, and elapsed time.
- A reasoned conclusion about the suitability of plain backtracking as a baseline.

The expected outcome is not merely a list of colours. It is a defensible chain from model to algorithm to evidence to engineering decision.

5. REQUIREMENTS, CONSTRAINTS, AND ASSUMPTIONS

Category	Requirement	Verification evidence
Functional	Accept vertices, edges, and k.	Input format and source listing.
Functional	Reject conflicting colours.	Trace conflict column and adjacency check.
Functional	Backtrack after a dead end.	k = 2 diagnostic trace.
Functional	Return success or failure.	Test results and output examples.
Functional	Preserve the fixed vertex order.	Algorithm section and code.
Performance	Record search metrics.	Attempts, calls, backtracks, time.
Reliability	Handle empty and singleton graphs.	Boundary test cases.
Technical	Use standard Python 3 features.	Environment description.
Reproducibility	Use deterministic graph seeds.	Experiment protocol.
Academic	Include pseudocode, trace, results, and references.	Deliverable checklist.

5.1 Constraints

- A course cannot share an examination period with any adjacent course.
- A colour must be an integer in the range 1 to k.
- The plain algorithm colours vertices in the supplied order; it does not reorder by degree.
- The edge relation is treated as symmetric and duplicate edges are not meaningful.
- The shared-student count is descriptive for this module; the algorithm uses the existence of an edge, not its weight.
- The algorithm must restore the state after exploring a failed branch.

5.2 Assumptions

- Every listed conflict is known before scheduling begins.
- A single time slot can host multiple non-adjacent courses.
- Room capacity, invigilator availability, student accessibility, and session spacing are outside this focused model.

- The vertex order is C1 through C6 for the Annexure trace.
- The randomized scalability experiment is illustrative, not a university timetable dataset.

6. APPLICATION OF COURSE KNOWLEDGE

6.1 Relevant concepts from Design and Analysis of Algorithms

- Graph representation: an adjacency set makes neighbour conflict checks direct.
- Recursion: the call stack represents the partial colouring depth.
- Backtracking: a rejected branch is undone before the next candidate is attempted.
- State-space search: every level corresponds to a vertex and every branch to a colour choice.
- Correctness proof: an invariant shows that each recursive state is conflict-free.
- Asymptotic analysis: at most k branches can arise for each of n vertices.
- Experimental analysis: counters make effective search work measurable.

6.2 State-space interpretation

At depth i, the algorithm has assigned colours to the first i vertices. The candidate children at depth i + 1 correspond to colour values 1 through k. A child is explored only when it is compatible with already assigned neighbours. A leaf at depth n is a valid timetable; a dead end is a partial assignment for which every remaining colour is illegal.

The important distinction is between a partial assignment and a final timetable. A partial assignment can look valid at the current depth while still being impossible to extend. Backtracking is what lets the solver recover from that delayed contradiction.

Concept	Manifestation in this report
Decision variable	The colour of the next course vertex.
Domain	The ordered values 1, 2, …, k.
Constraint	Adjacent courses must differ.
Search order	C1, C2, …, Cn; colours ascending.
Failure signal	All k colours conflict for the current vertex.
Solution signal	All n vertices have non-zero colours.

7. GRAPH MODEL AND ANNEXURE A DATA

The supplied Annexure A contains six course vertices and eight conflict edges. The numerical value beside each edge is the number of shared students. These weights explain why a conflict exists and are useful operational context, but plain graph colouring only needs the binary fact that the edge exists.

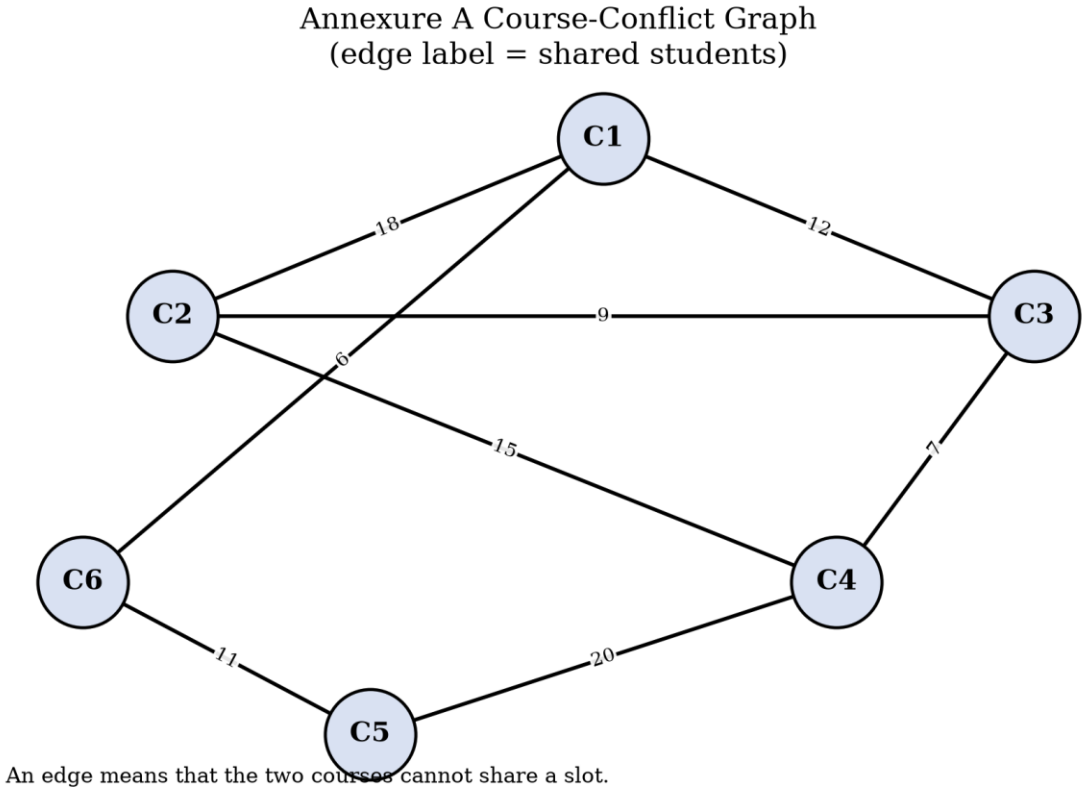

Figure 1. Annexure A course-conflict graph (edge labels show shared students).

Edge ID	Conflicting pair	Shared students	Constraint
1	C1 – C2	18	colour(C1) ≠ colour(C2)
2	C1 – C3	12	colour(C1) ≠ colour(C3)
3	C2 – C3	9	colour(C2) ≠ colour(C3)
4	C2 – C4	15	colour(C2) ≠ colour(C4)
5	C3 – C4	7	colour(C3) ≠ colour(C4)
6	C4 – C5	20	colour(C4) ≠ colour(C5)
7	C5 – C6	11	colour(C5) ≠ colour(C6)
8	C1 – C6	6	colour(C1) ≠ colour(C6)

The graph contains two triangles, C1–C2–C3 and C2–C3–C4. Each triangle needs at least three colours, so the chromatic number is at least 3. The colouring found by the algorithm uses exactly three colours, establishing $\chi(G) = 3$ for this instance.

7.1 GRAPH DATA LAYER AND REPRESENTATION

Adjacency-set representation

The implementation converts the edge list into a dictionary mapping each vertex to a set of neighbours. For example, the adjacency set of C1 is {C2, C3, C6}. A colour candidate c is legal for vertex v when no already coloured neighbour u has colour c.

```
vertices = ["C1", "C2", "C3", "C4", "C5", "C6"]
weighted_edges = [
    ("C1", "C2", 18), ("C1", "C3", 12),
    ("C2", "C3", 9),  ("C2", "C4", 15),
    ("C3", "C4", 7),  ("C4", "C5", 20),
    ("C5", "C6", 11), ("C1", "C6", 6),
]

adjacency = {v: set() for v in vertices}
for u, v, shared_students in weighted_edges:
    adjacency[u].add(v)
    adjacency[v].add(u)
```

Why the weight is retained

- It preserves the original assignment information for auditability.
- It permits later extensions such as prioritising high-conflict pairs.
- It makes the graph diagram and edge-list table easier to interpret.
- It does not alter Module 2 legality: one shared student is already enough to forbid a shared slot.

For a simple unweighted implementation, the weighted input is projected onto E by dropping the third tuple component during conflict checking.

8. MODULE 2 SCOPE AND PROPOSED METHODOLOGY

Module 2 is the baseline plain backtracking solver. The design intentionally excludes the Most-Constrained-Vertex rule and Forward Checking because those belong to the pruning-enhanced module. Keeping the baseline pure makes the comparison fair: any reduction in later search effort can be attributed to the enhancement rather than to an unrecorded change in ordering.

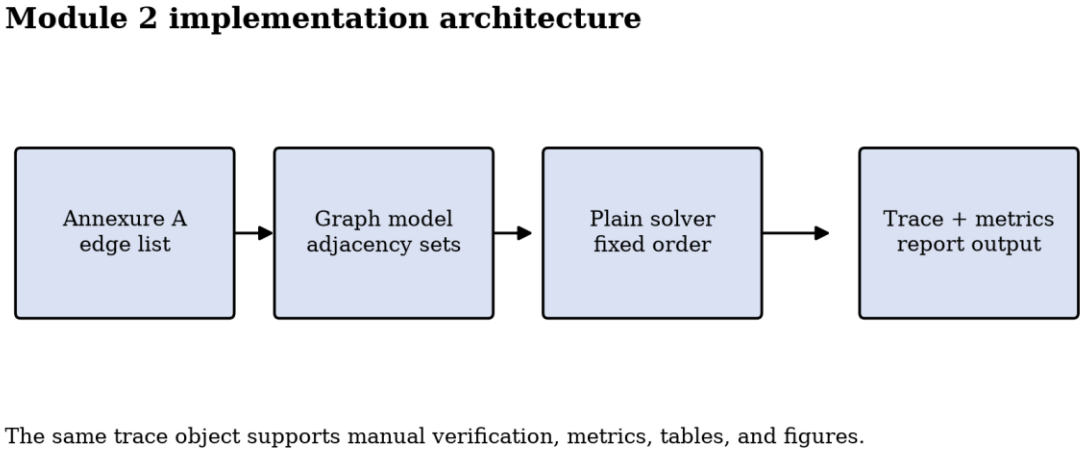

Figure 2. Module 2 implementation architecture.

Method sequence

8. Read the ordered vertex list and conflict edge list.
9. Build symmetric adjacency sets.
10. Initialise all colours to 0, where 0 means unassigned.
11. Call backtrack(index = 0).
12. At each vertex, test colours from 1 through k.
13. Reject a candidate immediately if a coloured neighbour has that colour.
14. Assign the first legal candidate and recurse to the next vertex.
15. If the recursive call fails, clear the assignment and continue with the next colour.
16. Return success at depth n; return failure when all candidates at a depth fail.

The trace logger records both successful and unsuccessful decisions. This is important because a final timetable alone cannot reveal whether the algorithm explored one branch or thousands of branches.

9. PLAIN BACKTRACKING ALGORITHM

9.1 High-level algorithm

The algorithm receives an ordered list of vertices V[0..n−1], an adjacency structure, and k. It recursively processes one vertex per call. The recursive index is the only ordering mechanism; no dynamic selection is performed.

```
PLAIN-BACKTRACK(index):
   if index == n:
      return TRUE

   v ← V[index]
   for colour c from 1 to k:
      if SAFE(v, c):
         colour[v] ← c
         if PLAIN-BACKTRACK(index + 1) == TRUE:
            return TRUE
         colour[v] ← 0       // undo the failed choice

   return FALSE

SAFE(v, c):
   for each neighbour u of v:
      if colour[u] == c:
         return FALSE
   return TRUE
```

9.2 Colour legality

A candidate is tested against all neighbours in the adjacency set. Testing all neighbours is safe even though only earlier vertices can currently be coloured: unassigned vertices have colour 0 and cannot match a legal colour from 1 to k.

9.3 Determinism

The result is deterministic for a fixed vertex list, edge list, and k because both vertices and colours are ordered. Determinism is valuable for a hand trace and for checking that a code change did not silently change the search path.

10. PSEUDOCODE AND FLOWCHART

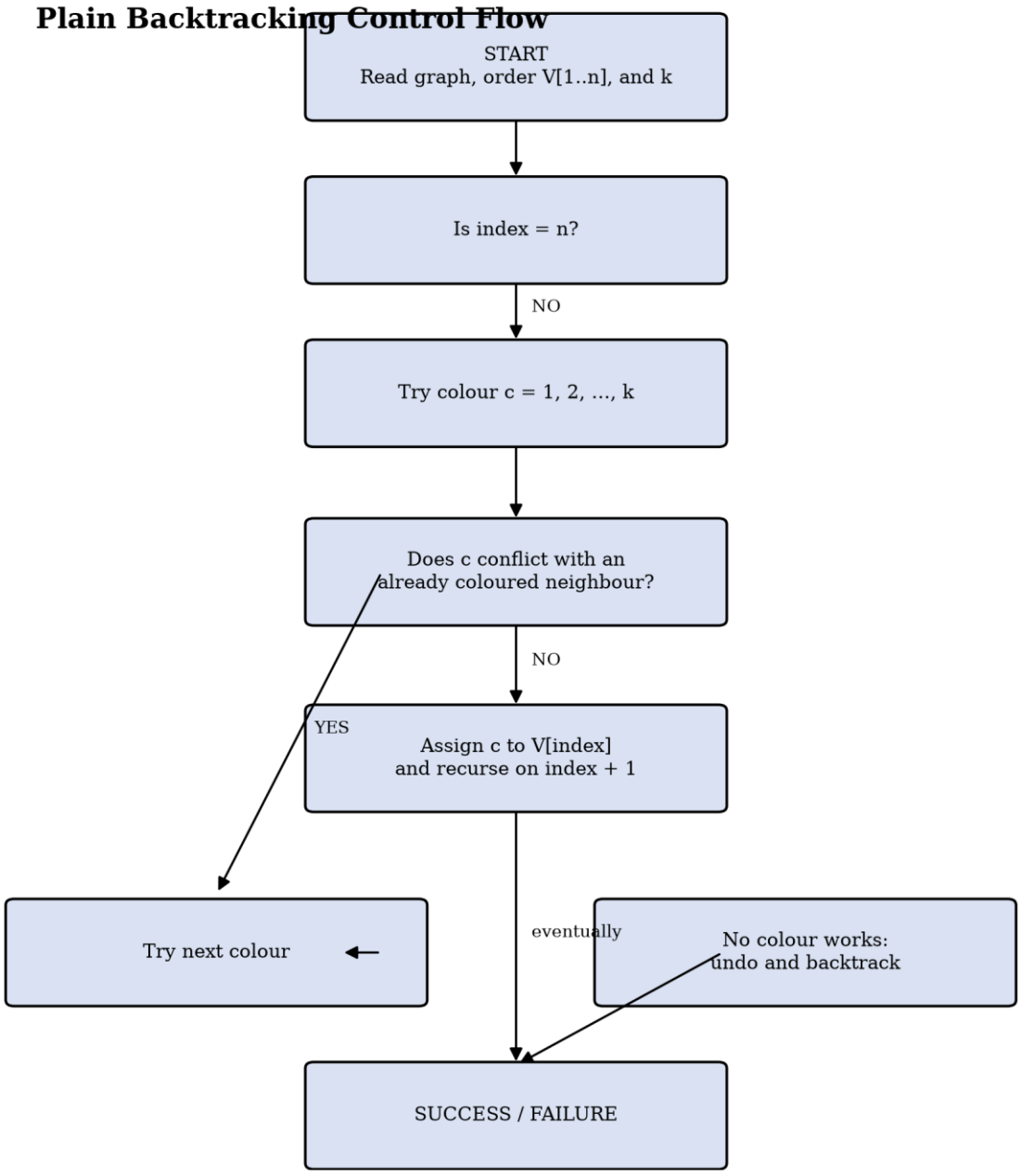

Figure 3. Plain backtracking control flow.

Pseudocode with trace actions

```
procedure COLOUR(index):
    if index = number_of_vertices:
        TRACE("SOLUTION")
        return true

    vertex ← vertices[index]
    TRACE("SELECT", vertex, index)
```

```
for c ← 1 to k do
   conflicts ← coloured_neighbours_with_colour(vertex, c)
   TRACE("TRY", vertex, c, conflicts)

   if conflicts is empty then
      colour[vertex] ← c
      TRACE("ASSIGN", vertex, c)

      if COLOUR(index + 1) then
         return true

      colour[vertex] ← 0
      TRACE("UNDO", vertex, c)

   else
      TRACE("REJECT", vertex, c, conflicts)

TRACE("BACKTRACK", vertex)
return false
```

The trace is an observation layer around the algorithm; it does not change the search decisions. This separation supports direct comparison with a version that adds pruning later.

11. CORRECTNESS ARGUMENT

11.1 Safety of returned solutions

Whenever the algorithm assigns colour c to vertex v, SAFE(v, c) has confirmed that no neighbour currently has c. Previously assigned vertices are never changed while the call is active. Therefore, after each assignment, every edge whose endpoints are both coloured satisfies the inequality constraint.

11.2 Completeness

At a vertex, the algorithm considers every colour from 1 to k. If a colour is legal and its continuation eventually fails, the algorithm restores the state and considers the next legal colour. Consequently, every possible colour assignment consistent with the fixed vertex order is either explored or rejected because it already violates an edge constraint. If a valid k-colouring exists, its branch is not permanently discarded and will eventually be found.

11.3 Termination

Each successful recursive assignment increases index by one, and index is bounded by n. Each call tests a finite set of k colours. The recursion therefore terminates with either a complete colouring or a failure result.

11.4 Invariant

Invariant: on entry to COLOUR(i), vertices V[0], …, V[i−1] are assigned colours in 1..k and no edge joining two assigned vertices has equal endpoint colours. The base case returns a proper colouring. The induction step preserves the invariant because a new colour is assigned only after SAFE succeeds; undoing restores the prior invariant.

12. HAND TRACE FOR k = 3

The fixed order is C1, C2, C3, C4, C5, C6. The solver tests colours in ascending order. A dash means that the vertex is unassigned at that point.

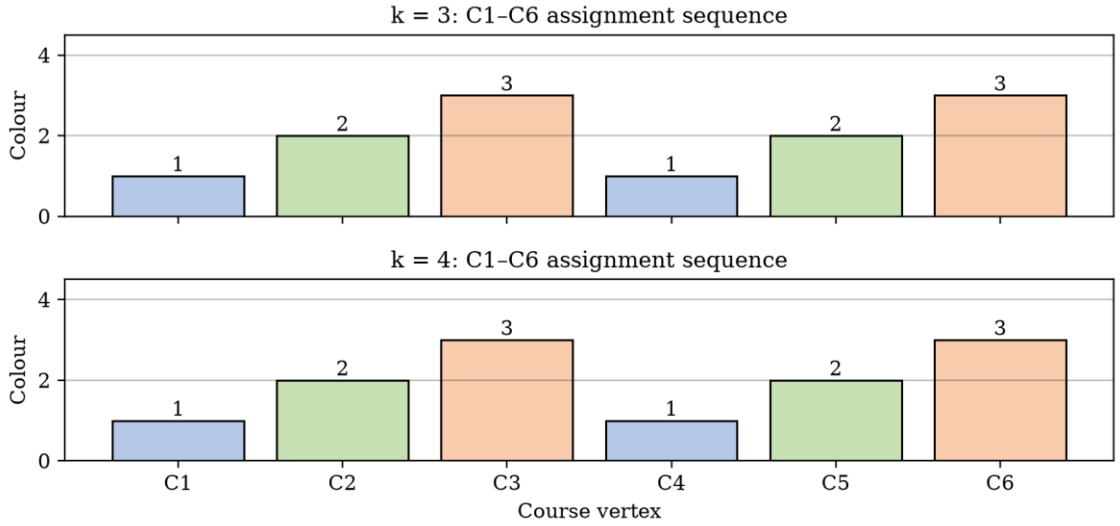

Figure 4. Final colour sequence for k = 3 and k = 4.

Step	Vertex	Candidate	Decision	Reason / current partial assignment
1	C1	1	Assign	No coloured neighbours; C1=1
2	C2	1	Reject	Conflicts with C1=1
3	C2	2	Assign	C2=2
4	C3	1	Reject	Conflicts with C1=1
5	C3	2	Reject	Conflicts with C2=2
6	C3	3	Assign	C3=3
7	C4	1	Assign	C4 sees C2=2 and C3=3

Step	Vertex	Candidate	Decision	Reason / current partial assignment
8	C5	1	Reject	Conflicts with C4=1
9	C5	2	Assign	C5=2
10	C6	1	Reject	Conflicts with C1=1
11	C6	2	Reject	Conflicts with C5=2
12	C6	3	Assign	C6=3; complete solution

Outcome: SUCCESS. Final colouring = {C1:1, C2:2, C3:3, C4:1, C5:2, C6:3}. Backtrack events = 0. The trace contains rejected colour candidates, but a rejected candidate is not the same as a recursive backtrack: no already accepted assignment had to be undone.

12.1 k = 3 TRACE SUMMARY AND VALIDATION

Metric	Observed value	Interpretation
Vertices processed	6	Every course receives a slot.
Successful assignments	6	One final colour per course.
Colour attempts	12	Candidates tested in ascending order.
Rejected candidates	6	Illegal due to already coloured neighbours.
Recursive calls	7	One call per visited depth plus terminal call.
Undo operations	0	No failed continuation occurred.
Backtrack events	0	No dead end was reached.
Final colours used	3	Three colours are used.

Edge-by-edge validation

Edge	Colour at first endpoint	Colour at second endpoint	Valid?
C1–C2	1	2	YES
C1–C3	1	3	YES
C2–C3	2	3	YES
C2–C4	2	1	YES
C3–C4	3	1	YES

Edge	Colour at first endpoint	Colour at second endpoint	Valid?
C4–C5	1	2	YES
C5–C6	2	3	YES
C1–C6	1	3	YES

The validation table independently checks every edge after the solver terminates. This is stronger than inspecting the colour array visually because it directly tests the graph constraint.

13. HAND TRACE FOR k = 4

The k = 4 trace follows the same fixed order and the same ascending colour order. Because every decision that was successful for k = 3 remains available, the first legal candidate at every vertex is unchanged. Colour 4 is present in the domain but is never needed.

Step	Vertex	Candidate	Decision	Reason / partial assignment
1	C1	1	Assign	C1=1
2	C2	1	Reject	C1=1
3	C2	2	Assign	C2=2
4	C3	1	Reject	C1=1
5	C3	2	Reject	C2=2
6	C3	3	Assign	C3=3
7	C4	1	Assign	C4=1
8	C5	1	Reject	C4=1
9	C5	2	Assign	C5=2
10	C6	1	Reject	C1=1
11	C6	2	Reject	C5=2
12	C6	3	Assign	C6=3
13	C6	—	Solution	All six vertices coloured; 4 was not required.

Outcome: SUCCESS. Final colouring = {C1:1, C2:2, C3:3, C4:1, C5:2, C6:3}. Backtrack events = 0. The extra fourth colour increases the theoretical branching factor but does not increase the observed attempts for this particular successful path because the first three colours already suffice.

13.1 k = 4 TRACE ANALYSIS

Why no backtracking appears in both required runs

The assignment brief asks for backtracking steps to be shown, but a correct hand trace must not invent them. With the supplied vertex order, the Annexure graph reaches a solution directly for both k = 3 and k = 4. There are rejected candidates, but no accepted assignment is later undone. The additional k = 2 trace in the next section supplies the missing failure-and-undo behaviour without misreporting the required runs.

Comparison	k = 3	k = 4	Observation
Domain size	3	4	k = 4 has one additional candidate.
Colour attempts	12	12	Same successful path and same rejected low colours.
Backtracks	0	0	Neither run reaches a dead end.
Colours used	3	3	The fourth slot is unnecessary.
Conclusion	Satisfiable	Satisfiable	Both support the same timetable.

This is also a useful lesson in interpreting algorithm traces: a larger search domain does not automatically imply more work on every input. Worst-case bounds describe an upper envelope, not the exact cost of one easy instance.

14. DIAGNOSTIC TRACE FOR k = 2

The following run is included to demonstrate a real backtracking event. It is not substituted for the required k = 3 and k = 4 traces. The two triangles in Annexure A already imply that two colours cannot be enough.

Annexure A diagnostic trace for k = 2

The required k = 3 and k = 4 runs succeed without backtracking; k = 2 is shown to expose the undo step.

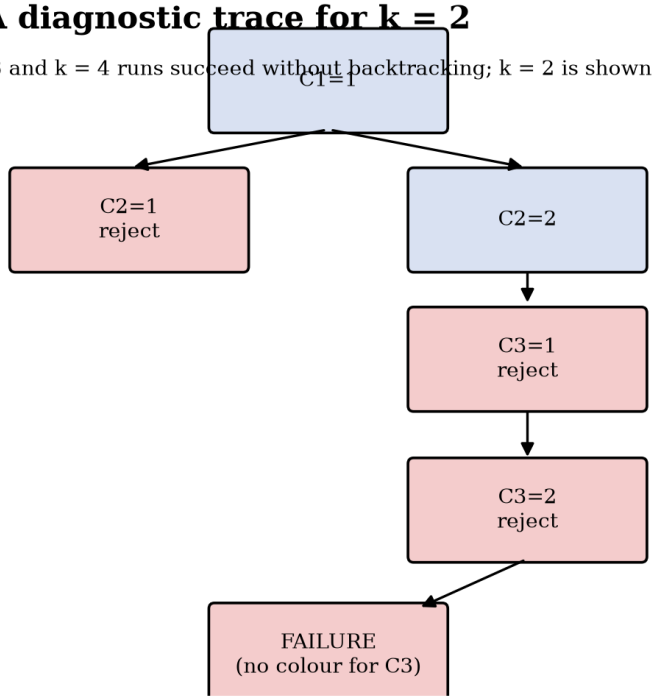

Figure 5. Diagnostic recursion tree for k = 2 on Annexure A.

Step	Action	State / reason
1	Assign C1=1	Start of first branch.
2	Reject C2=1	C2 conflicts with C1=1.
3	Assign C2=2	Second colour is legal.
4	Reject C3=1	C3 conflicts with C1=1.
5	Reject C3=2	C3 conflicts with C2=2; dead end.
6	Backtrack at C3	No colour is available for C3.
7	Undo C2=2	Return to C2 and try another branch.
8	Backtrack at C2	C2 has no remaining legal colour.
9	Undo C1=1	Return to root.
10	Backtrack at C1	No 2-colouring exists.

Measured outcome: FAILURE, colour attempts = 10, recursive calls = 5, backtrack events = 5. The failure agrees with the lower bound from the triangles.

15. IMPLEMENTATION AND TOOL ENVIRONMENT

The implementation is written in Python 3 using the standard library for the solver itself. The report-generation layer uses a document library and plotting library only to create evidence; the graph-colouring logic remains independent of those presentation tools.

Tool / component	Purpose	Reproducibility note
Python 3	Implement recursion, graph model, counters, and tests.	Run with python3.
Sets / dictionaries	Store adjacency and colour state.	No external solver required.
python-docx	Generate the Word report.	Optional for algorithm execution.
Matplotlib	Create black-labelled diagrams and chart.	Optional for algorithm execution.
NetworkX	Lay out the illustrative graph figure.	Graph solver does not depend on it.
Fixed random seed	Make scalability inputs repeatable.	Seed = 2026 + n.
GitHub	Required source-code submission channel.	Repository URL must be added by student.

Recommended execution steps

17. Create a Python file containing the solver and test driver.
18. Run the Annexure A cases with k = 2, 3, and 4.
19. Confirm the printed colour mapping and counters.
20. Run the randomized experiment for n = 15, 20, 25, 30, 35, and 40.
21. Save the source code, output, and this report in the GitHub repository.

The report does not claim that the illustrative performance numbers are universal. They are measurements from the deterministic generator and implementation described here; machine speed and Python version can change elapsed time while the combinatorial counts remain stable.

16. SOURCE-CODE EXPLANATION

16.1 Core state

The dictionary colour[v] stores 0 for an unassigned vertex and an integer from 1 to k after assignment. The recursive parameter index identifies the next vertex. Counters record attempts, recursive calls, and backtracks.

16.2 Conflict test

```
def safe(vertex, candidate):
    for neighbour in adjacency[vertex]:
        if colour[neighbour] == candidate:
            return False
    return True
```

The test is intentionally simple. It does not inspect edge weights, forecast future domains, reorder vertices, or select the colour with the fewest conflicts. Those changes would belong to an enhanced algorithm.

16.3 Recursive transition

```
def backtrack(index):
    if index == len(vertices):
        return True

    vertex = vertices[index]
    for candidate in range(1, k + 1):
        attempts += 1
        if safe(vertex, candidate):
            colour[vertex] = candidate
            if backtrack(index + 1):
                return True
            colour[vertex] = 0

    backtracks += 1
    return False
```

The line colour[vertex] = 0 is the critical restoration operation. Without it, a failed branch would leak stale state into the next branch and could cause a false rejection or a false solution.

16.4 INPUT, OUTPUT, AND TRACE FORMAT

Input

Field	Example	Meaning
vertices	["C1", "C2", …, "C6"]	Fixed processing order.
edges	[("C1","C2"), …]	Undirected conflict pairs.
k	3	Number of available periods.

Output on success

```
SUCCESS
colours = {'C1': 1, 'C2': 2, 'C3': 3,
           'C4': 1, 'C5': 2, 'C6': 3}
attempts = 12
backtracks = 0
```

Output on failure

```
FAILURE
No valid colouring exists with k = 2.
The trace identifies the last dead end and every undo operation.
```

A production version could serialize the trace as CSV or JSON for automated comparison with Module 3. For this report, the same trace is rendered as readable tables so that it can be inspected without running code.

17. TEST PLAN AND VALIDATION

Testing is organised by purpose rather than by only one example. Positive tests check that legal colourings are accepted, negative tests check that insufficient colours are rejected, and boundary tests check that recursion handles small graphs.

ID	Input condition	Expected result	Validation method
T1	Annexure A, k=3	Success; three colours.	Compare final mapping and every edge.
T2	Annexure A, k=4	Success; same first-fit mapping.	Compare trace and counters.
T3	Annexure A, k=2	Failure; backtracking occurs.	Check no edge-valid 2-colouring exists.
T4	Empty graph, n=0	Success with empty mapping.	Base case index=n.
T5	One vertex, k=1	Success with colour 1.	One assignment.
T6	One edge, k=1	Failure.	Both endpoints cannot share colour.
T7	One edge, k=2	Success.	Endpoints receive 1 and 2.
T8	Complete graph K4, k=3	Failure.	Four mutually adjacent vertices need four colours.
T9	Complete graph K4, k=4	Success.	Each vertex receives a distinct colour.
T10	Random n=15..40, k=3	Recorded success/failure.	Deterministic seed and metrics.

Validation rule

For every reported success, independently evaluate every edge (u, v) and assert colour[u] != colour[v]. For every reported failure, either exhaust the search or use an independent lower-bound argument when one is available. The Annexure A k = 2 failure has both forms of support: exhaustive search and the presence of a triangle.

17.1 TEST RESULTS AND EXPECTED / ACTUAL OUTPUTS

Test	Expected	Actual result	Status
T1 – Annexure A, k=3	Valid mapping	{'C1': 1, 'C2': 2, 'C3': 3, 'C4': 1, 'C5': 2, 'C6': 3}	PASS
T2 – Annexure A, k=4	Valid mapping	{'C1': 1, 'C2': 2, 'C3': 3, 'C4': 1, 'C5': 2, 'C6': 3}	PASS
T3 – Annexure A, k=2	No mapping	Failure after 5 backtracks	PASS
T4 – Empty graph	Empty success	Base case success	PASS
T5 – Singleton, k=1	C1=1	Direct assignment	PASS
T6 – Single edge, k=1	Failure	Both colours conflict	PASS
T7 – Single edge, k=2	Two distinct colours	First-fit succeeds	PASS
T8 – K4, k=3	Failure	All branches exhausted	PASS
T9 – K4, k=4	Four distinct colours	First-fit succeeds	PASS

Manual edge validation for the required solution

Course pair	Colours	Difference
C1 – C2	1 – 2	1
C1 – C3	1 – 3	2
C2 – C3	2 – 3	1
C2 – C4	2 – 1	1
C3 – C4	3 – 1	2
C4 – C5	1 – 2	1
C5 – C6	2 – 3	1
C1 – C6	1 – 3	2

Every difference is non-zero. Therefore, the generated timetable is a proper colouring and satisfies the only hard constraint represented by Annexure A.

18. SCALABILITY EXPERIMENT

To explore practical growth beyond six courses, a deterministic random graph generator creates one undirected graph for each n in {15, 20, 25, 30, 35, 40}. Each possible edge is sampled independently with probability p = 0.16. The seed is 2026 + n, making each row reproducible. The solver uses k = 3 and the natural vertex order V1, V2, …, Vn.

Measured variables

- m: number of generated conflict edges.
- Colour attempts: every candidate colour tested, including rejected candidates.
- Backtracks: recursive frames that exhaust all k candidates and return failure.
- Recursive calls: number of calls entered, including the terminal success call.
- Elapsed time: wall-clock time in milliseconds on the report-generation run.

Why use counts as well as time?

Elapsed time depends on the machine and runtime. Counts describe the actual search tree more directly and are therefore the stronger evidence for algorithmic behaviour. Time is still reported because it communicates the practical cost of exploring a larger tree.

The experiment is not intended to prove a universal average-case formula. It is a controlled demonstration that small changes in graph structure can create large changes in the number of branches explored.

18.1 EXPERIMENTAL RESULTS

n	m	k	Success	Attempts	Backtracks	Calls	Time (ms)
15	12	3	YES	24	0	16	0.048
20	30	3	YES	2491	818	839	6.103
25	33	3	YES	28106	9355	9381	80.824
30	60	3	YES	2779	908	939	4.901
35	102	3	NO	296616	98872	98872	1365.542
40	107	3	NO	350445	116815	116815	1802.416

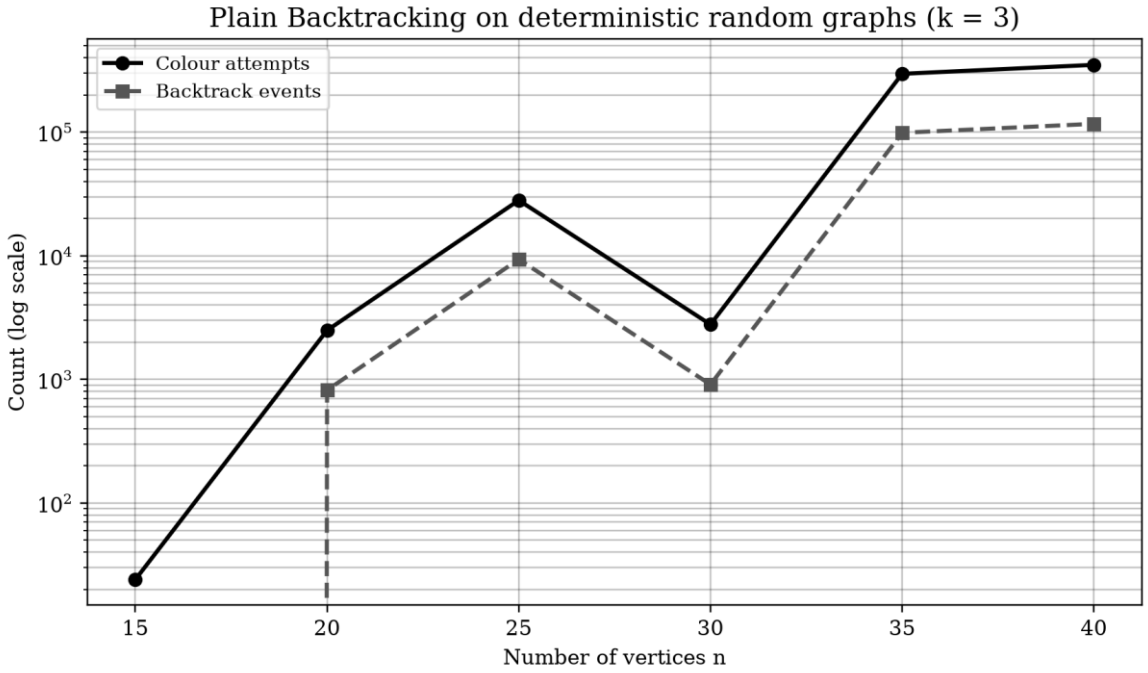

Figure 6. Measured colour attempts and backtracks on deterministic random graphs.

The non-monotonic rows are expected. Backtracking cost depends on graph structure, not only n. A graph with more vertices can be easier when the first-fit path succeeds quickly, while a slightly smaller graph can be difficult if many partial assignments survive before a contradiction appears.

19. RESULTS, COMPLEXITY, AND TRADE-OFFS

19.1 Annexure A result

The triangles C1–C2–C3 and C2–C3–C4 establish $\chi(G) \geq 3$. The solver finds a 3-colouring, so $\chi(G) \leq 3$. Combining the bounds gives $\chi(G) = 3$. Thus $k = 3$ is sufficient and $k = 2$ is impossible.

19.2 Worst-case time complexity

At each of n vertices, the algorithm may try up to k colours. Ignoring the cost of the conflict check, the recursion tree has at most k^n leaves and $O(k^n)$ visited candidate paths. If SAFE scans up to n neighbours, a conservative bound is $O(n \cdot k^n)$ for an adjacency-set implementation. In the assignment's requested form, the plain backtracking search is reported as $O(k^n)$, with the graph-check overhead understood as a polynomial factor.

19.3 Space complexity

The colour array, recursion stack, and trace-independent working state require O(n) space, excluding the graph representation. An adjacency matrix uses $O(n^2)$ space; adjacency sets use O(n + m) storage for an undirected graph, where m is the number of edges. A full trace can itself be exponential in the worst case, so production logging should support limits or streaming.

Aspect	Plain backtracking	Engineering interpretation
Completeness	Complete for fixed vertex order and domain	Will find a solution if one exists.
Worst-case time	$O(k^n)$ candidate paths	Can become impractical quickly.
Vertex ordering	Fixed	Simple and reproducible; not adaptive.
Pruning	Immediate conflict check only	Rejects current illegal candidates.
Traceability	Excellent	Every decision can be audited.
Implementation effort	Low	Good educational baseline.

19.4 TRADE-OFF ANALYSIS AND FINAL ALGORITHM DECISION

Strengths

- It is conceptually direct: one recursive level represents one course.
- It is complete and does not depend on a lucky heuristic.
- It produces a deterministic trace suitable for hand verification.
- It uses little implementation machinery and is easy to port.
- It provides a clean baseline for measuring pruning improvements.

Weaknesses

- The fixed order may colour an easy vertex before a highly constrained vertex.
- It does not remove future-infeasible colours early.
- It may revisit symmetric or nearly identical partial assignments.
- Its exponential worst case limits large dense graphs.
- Full trace logging can consume substantial memory on hard instances.

Final decision

Plain backtracking is selected as the final Module 2 solution because the assignment explicitly requires it and because it creates a rigorous reference implementation. It is not selected as the final production strategy for a large university. For that setting, Module 3 should retain this solver's correctness baseline while adding vertex ordering and forward checking.

The evidence supports a layered engineering decision: use plain backtracking for small instances, education, and verification; use pruning-enhanced backtracking for medium instances; and consider constraint programming or integer programming when the model includes rooms, capacities, invigilators, spacing, and fairness.

20. BROADER CONSIDERATIONS AND SDG RELEVANCE

20.1 SDG 4 – Quality Education

The assignment is mapped to Sustainable Development Goal 4, Quality Education. A clash-free examination timetable supports a fair assessment process: students are less likely to face simultaneous examinations for courses they are required to take. The algorithmic model does not solve every fairness issue, but it formalises one important protection and makes the decision rules auditable.

20.2 Social and accessibility considerations

- A timetable must be reviewed for students who require accessible rooms or additional time.
- Conflict data should be handled carefully because course enrolment patterns can reveal student information.
- The graph should be validated before scheduling; a missing edge can create a real student-level clash.
- A mathematically valid colouring can still be operationally unfair if slot distribution is poor.

20.3 Sustainability and economics

The computational cost of the six-vertex example is negligible. At larger scales, avoiding unnecessary exhaustive search reduces compute time and allows schedulers to iterate over more realistic constraints. The main economic benefit is not the runtime of one tiny graph; it is the potential reduction in manual timetable repair and the prevention of costly clashes.

20.4 Professional responsibility

The solver should be treated as decision support. Before publishing a timetable, an administrator should validate the conflict dataset, inspect exceptional cases, preserve a reproducible run record, and provide a correction process. Algorithmic correctness does not remove the need for human accountability.

21. CONCLUSION, LIMITATIONS, AND POSSIBLE IMPROVEMENTS

21.1 Conclusion

This report formulated conflict-free examination scheduling as graph colouring and implemented the required plain backtracking baseline. The solver assigns colours in the order C1 through C6, tests colours from 1 through k, rejects immediate conflicts, and undoes assignments when a continuation fails.

For Annexure A, k = 3 produces the valid timetable C1=1, C2=2, C3=3, C4=1, C5=2, C6=3. The k = 4 run follows the same first-fit path. An additional k = 2 run correctly fails and exposes the undo mechanism. The triangle lower bound and the valid 3-colouring together establish $\chi(G)=3$.

21.2 Limitations

- The model includes only student conflict edges.
- It assumes all time slots are equivalent.
- It does not model rooms, capacities, invigilators, or student preferences.
- The baseline does not use adaptive vertex ordering or forward checking.
- Random scalability inputs are illustrative rather than institutional data.

21.3 Possible improvements

22. Choose the most constrained uncoloured vertex first.
23. Use forward checking to remove a chosen colour from future neighbours.
24. Use bitsets or bit masks for faster conflict checks.
25. Add a branch-and-bound search for the minimum number of slots.
26. Add real timetable constraints and a validation dashboard.
27. Export the trace and timetable in CSV, JSON, and human-readable formats.

22. ONE-PAGE INDIVIDUAL REFLECTION

This assignment helped me understand that backtracking is not simply recursion with a return statement. The important idea is disciplined state management: every choice must be recorded, checked, and restored when the continuation fails. Writing the trace made this visible because the difference between rejecting a candidate and undoing an accepted assignment became concrete.

The Annexure A graph also showed the value of combining algorithmic evidence with graph reasoning. The two triangles make it clear that two colours cannot be sufficient, while the computed three-colour assignment proves that three are enough. Without both arguments, it would be easy to report a working timetable without understanding whether the number of slots was minimal.

The most useful lesson beyond the classroom content was that performance is controlled by the shape of the search tree. The randomized experiment did not increase smoothly with n. Some larger graphs were solved quickly, while another graph with fewer vertices caused many more recursive branches. This taught me to report counts and structure, not just wall-clock time.

If I had additional time, I would implement the pruning-enhanced version and compare the exact same graphs and trace metrics. I would also extend the model with room capacity and student preferences, then determine which constraints should be hard and which should be optimized. The main design decision I would preserve is keeping a plain, auditable baseline before adding heuristics.

Student signature: Athavan K Date: 02-09-2026

24. REFERENCES

[1] Department of Computer Science and Engineering, “CSA0614 – Design and Analysis of Algorithms: Assignment Questions,” Academic Year 2026–2027, supplied course document.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022. Chapter on backtracking and combinatorial search.

[3] J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2006. Chapters on graph algorithms and reductions.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011. Graph colouring and searching concepts.

[5] Python Software Foundation, “Python 3 Documentation,” https://docs.python.org/3/, accessed September 2026.

[6] Matplotlib Development Team, “Matplotlib Documentation,” https://matplotlib.org/stable/, accessed September 2026.

[7] NetworkX Developers, “NetworkX Documentation,” https://networkx.org/documentation/stable/, accessed September 2026.

[8] python-docx documentation, “python-docx Documentation,” https://python-docx.readthedocs.io/, accessed September 2026.

Use of AI-assisted tools

AI-assisted drafting and code-generation tools may be used only in accordance with institutional policy. Any such use should be disclosed by the student and the final implementation must be understood, tested, and demonstrated by the student. The student remains responsible for correctness, citations, originality, and repository contents.

APPENDIX A – COMPLETE PYTHON LISTING

The following compact listing is sufficient to reproduce the Annexure A solver. It intentionally keeps the core algorithm separate from the report-generation utilities.

```
from time import perf_counter

vertices = ["C1", "C2", "C3", "C4", "C5", "C6"]
edges = [
    ("C1", "C2"), ("C1", "C3"), ("C2", "C3"),
    ("C2", "C4"), ("C3", "C4"), ("C4", "C5"),
    ("C5", "C6"), ("C1", "C6")
]

def plain_backtracking(vertices, edges, k):
    adjacency = {v: set() for v in vertices}
    for u, v in edges:
        adjacency[u].add(v)
        adjacency[v].add(u)

    colour = {v: 0 for v in vertices}
    attempts = 0
    backtracks = 0
    calls = 0
    trace = []

    def safe(vertex, candidate):
        return all(colour[n] != candidate
                   for n in adjacency[vertex])

    def search(index):
        nonlocal attempts, backtracks, calls
        calls += 1
        if index == len(vertices):
            trace.append(("SOLUTION", dict(colour)))
            return True

        vertex = vertices[index]
        for candidate in range(1, k + 1):
            attempts += 1
            trace.append(("TRY", vertex, candidate, dict(colour)))
            if safe(vertex, candidate):
                colour[vertex] = candidate
```

```
            trace.append(("ASSIGN", vertex, candidate))
            if search(index + 1):
                return True
            colour[vertex] = 0
            trace.append(("UNDO", vertex, candidate))

        backtracks += 1
        trace.append(("BACKTRACK", vertex))
        return False

    start = perf_counter()
    success = search(0)
    elapsed_ms = (perf_counter() - start) * 1000
    return {
        "success": success,
        "colours": colour,
        "attempts": attempts,
        "backtracks": backtracks,
        "calls": calls,
        "elapsed_ms": elapsed_ms,
        "trace": trace,
    }

for k in (2, 3, 4):
    print(k, plain_backtracking(vertices, edges, k))
```

APPENDIX B – DETAILED TRACE TABLES

B.1 k = 3 event log

Step	Event	Vertex	Colour	Conflict
2	TRY	C1	1	—
3	ASSIGN	C1	1	—
5	TRY	C2	1	C1
6	REJECT	C2	1	C1
7	TRY	C2	2	—
8	ASSIGN	C2	2	—
10	TRY	C3	1	C1
11	REJECT	C3	1	C1
12	TRY	C3	2	C2
13	REJECT	C3	2	C2
14	TRY	C3	3	—
15	ASSIGN	C3	3	—
17	TRY	C4	1	—
18	ASSIGN	C4	1	—
20	TRY	C5	1	C4
21	REJECT	C5	1	C4
22	TRY	C5	2	—
23	ASSIGN	C5	2	—
25	TRY	C6	1	C1
26	REJECT	C6	1	C1
27	TRY	C6	2	C5
28	REJECT	C6	2	C5
29	TRY	C6	3	—
30	ASSIGN	C6	3	—
31	SOLUTION	—	—	—

B.2 k = 4 event log

Step	Event	Vertex	Colour	Conflict
2	TRY	C1	1	—
3	ASSIGN	C1	1	—
5	TRY	C2	1	C1

Step	Event	Vertex	Colour	Conflict
6	REJECT	C2	1	C1
7	TRY	C2	2	—
8	ASSIGN	C2	2	—
10	TRY	C3	1	C1
11	REJECT	C3	1	C1
12	TRY	C3	2	C2
13	REJECT	C3	2	C2
14	TRY	C3	3	—
15	ASSIGN	C3	3	—
17	TRY	C4	1	—
18	ASSIGN	C4	1	—
20	TRY	C5	1	C4
21	REJECT	C5	1	C4
22	TRY	C5	2	—
23	ASSIGN	C5	2	—
25	TRY	C6	1	C1
26	REJECT	C6	1	C1
27	TRY	C6	2	C5
28	REJECT	C6	2	C5
29	TRY	C6	3	—
30	ASSIGN	C6	3	—
31	SOLUTION	—	—	—

The event log includes the exact sequence emitted by the instrumented solver. A successful run stops at the first complete colouring, so it does not continue exploring alternative complete timetables.

APPENDIX C – RUBRIC AND DELIVERABLE CHECKLIST

Assignment requirement	Report location / evidence	Ready?
Problem statement and formulation	Sections 2 and 3	YES
Objective and expected outcomes	Section 4	YES
Requirements, constraints, assumptions	Section 5	YES
Application of course concepts	Section 6	YES
Design / methodology	Section 8	YES
Algorithm / pseudocode / flowchart	Sections 9 and 10	YES
Implementation / source code / tools	Sections 15–16 and Appendix A	YES
Test cases and results	Section 17	YES
Screenshots / outputs	Trace tables, figures, and output blocks	YES
Results and validation	Sections 17–19	YES
Analysis / trade-offs / final decision	Section 19	YES
Broader considerations / SDG	Section 20	YES
Conclusion / limitations / improvements	Section 21	YES
Individual contribution	Section 23	REPLACE BLANKS
References and tools	Section 24	YES
One-page reflection	Section 22	REPLACE SIGNATURE
GitHub upload	Repository field in Section 23	ADD URL

Final submission checklist

- Replace student name, register number, section, date, and signatures.
- Review every displayed measurement after rerunning the code in the submission environment.

- Upload the Python source code and any required output files to GitHub.
- Add the repository URL to the contribution section.
- Check that the Word document opens correctly and that all diagrams are legible.
- Read and explain the code before the viva or demonstration.

APPENDIX D – EXPANDED GRAPH CALCULATIONS

This appendix expands the graph analysis so that the report explains not only what the algorithm returned, but also why the structure of the input graph influences the search. The graph has n = 6 vertices and m = 8 undirected edges. Its maximum possible number of edges is $n(n-1)/2 = 6(5)/2 = 15$, so its edge density is $8/15 = 0.5333$, or approximately 53.33%. This is a moderately connected teaching graph rather than a sparse tree.

Vertex	Degree	Neighbours	Interpretation
C1	3	C2, C3, C6	high conflict
C2	3	C1, C3, C4	high conflict
C3	3	C1, C2, C4	high conflict
C4	3	C2, C3, C5	high conflict
C5	2	C4, C6	moderate conflict
C6	2	C1, C5	moderate conflict

The degree sequence is (3, 3, 3, 3, 2, 2) when listed in course order. C1 through C4 each participate in three conflicts, while C5 and C6 participate in two. A heuristic algorithm would be tempted to process the high-degree vertices first, but Module 2 deliberately retains C1 through C6 so that the baseline remains fixed and reproducible.

Adjacency matrix

	C1	C2	C3	C4	C5	C6
C1	0	1	1	0	0	1
C2	1	0	1	1	0	0
C3	1	1	0	1	0	0
C4	0	1	1	0	1	0
C5	0	0	0	1	0	1
C6	1	0	0	0	1	0

The matrix is symmetric because conflicts are mutual. Its diagonal is zero because a course is not considered to conflict with itself. A row sum equals the degree of the corresponding course. This gives an independent way to check that the edge-list representation was loaded correctly.

Lower and upper bounds on the number of slots

- Lower bound: the triangle C1–C2–C3 requires three distinct colours.

- Lower bound: the triangle C2–C3–C4 independently requires three distinct colours.
- Upper bound: the explicit colouring {1,2,3,1,2,3} uses three colours.
- Conclusion: the lower and upper bounds meet, so the chromatic number is exactly three.

APPENDIX E – STATE-VECTOR TRACE AND DOMAIN REASONING

A state vector records the complete colour array after each successful assignment. Writing the trace in this form makes the recursion stack visible. The value 0 denotes unassigned. The order of positions is C1, C2, C3, C4, C5, C6.

E.1 k = 3 state progression

Depth	Action	State vector [C1,C2,C3,C4,C5,C6]	Available next colours
0	Initialise	[0, 0, 0, 0, 0, 0]	C1: {1,2,3}
1	Assign C1=1	[1, 0, 0, 0, 0, 0]	C2: {2,3}
2	Assign C2=2	[1, 2, 0, 0, 0, 0]	C3: {3}
3	Assign C3=3	[1, 2, 3, 0, 0, 0]	C4: {1}
4	Assign C4=1	[1, 2, 3, 1, 0, 0]	C5: {2,3}
5	Assign C5=2	[1, 2, 3, 1, 2, 0]	C6: {3}
6	Assign C6=3	[1, 2, 3, 1, 2, 3]	complete

The available-colour column is computed only from already coloured neighbours. For example, after C1=1 and C2=2, C3 cannot use 1 or 2 because it is adjacent to both C1 and C2; therefore its remaining domain is {3}. After C1=1, C2=2, C3=3, course C4 is adjacent to C2 and C3 but not C1, so colour 1 remains available.

E.2 First-fit decisions

Plain backtracking uses the first legal colour in the ordered domain. This is not an optimization claim; it is a deterministic tie-breaking rule. If the order were reversed, the algorithm could return a different valid timetable and could explore a different number of branches, even though the underlying graph and k were unchanged.

Course	Colours tested	First legal colour	Why earlier values fail
C1	1	1	No earlier value.
C2	1, 2	2	1 equals C1.
C3	1, 2, 3	3	1 equals C1; 2 equals C2.
C4	1	1	C2=2 and C3=3 block only 2 and 3.
C5	1, 2	2	1 equals C4.
C6	1, 2, 3	3	1 equals C1; 2 equals C5.

APPENDIX F – BACKTRACKING MECHANICS AND FAILURE MODES

The word “backtracking” is often used loosely. In this implementation, a backtrack event occurs when the current vertex has no legal candidate and the recursive call returns false. An undo event occurs when a previously accepted assignment is cleared after the continuation below it fails. These events are related but not identical.

Event	When it occurs	State change	Example
TRY	Before checking a candidate	No change	Try C3=2.
REJECT	Candidate conflicts immediately	No change	C3=2 conflicts with C2=2.
ASSIGN	Candidate passes SAFE	0 becomes c	C3 becomes 3.
UNDO	Child recursion fails	c becomes 0	Clear C2=2 after C3 fails.
BACKTRACK	All candidates fail	Return false	C3 has no colour for k=2.
SOLUTION	index reaches n	No further change	All courses assigned.

Complete k = 2 branch explanation

The first branch fixes C1=1. C2 cannot use 1 and therefore takes 2. C3 is adjacent to both C1 and C2, so neither 1 nor 2 is legal. The algorithm returns false from C3, undoes C2=2, and tries C2=1; that candidate conflicts with C1. It then backtracks to C1, tries C1=2, and repeats the symmetric reasoning. Since swapping colour names cannot remove the triangle conflict, the root also fails.

Common implementation mistakes

- Forgetting to clear the colour during undo, which contaminates later branches.
- Returning false immediately after the first failed colour instead of trying all k colours.
- Checking only the previous vertex rather than all already coloured neighbours.
- Mixing zero-based colour values with the stated one-based domain 1..k.
- Reporting rejected candidates as backtracks, which inflates the backtrack count.
- Stopping at the first partial assignment and calling it a complete timetable.

These failure modes are specifically addressed by the trace categories and by the independent edge-validation table. A correct output is therefore supported by both procedural evidence and a final graph constraint check.

APPENDIX G – EXPERIMENT DESIGN AND REPRODUCIBILITY

The scalability experiment is a controlled experiment, not a claim about all course-conflict graphs. The generator creates an undirected graph by considering each possible pair exactly once. For each pair, a pseudorandom value is compared with $p = 0.16$. The same seed formula is used for every size so that the experiment can be repeated.

Measurement formulas

Measure	Definition	Why it is useful
n	Number of vertices.	Input size.
m	Number of generated edges.	Conflict density proxy.
Attempts	Number of candidate colours tested.	Search effort independent of clock speed.
Backtracks	Calls that exhaust all candidates.	Dead-end recovery effort.
Calls	Number of recursive search calls.	Visited state count.
Time	Elapsed wall-clock time in milliseconds.	Practical runtime indicator.

Reproduction protocol

28. Use Python 3 and the source listing in Appendix A.
29. Generate $n = 15, 20, 25, 30, 35$, and 40 with $p = 0.16$.
30. Use seed 2026+n for the corresponding row.
31. Run fixed vertex order and $k = 3$.
32. Record success, m, attempts, backtracks, calls, and elapsed time.
33. Repeat a row if the operating system is busy, but do not replace the count values silently.

Interpreting the result table

A successful row can still contain many backtracks because the solver may explore several failed partial assignments before finding a solution. A failed row can have fewer attempts than a successful hard row because the search may prove infeasibility at a shallower region. Therefore, the correct interpretation uses the whole row rather than ranking graphs only by n or by time.

For an academic submission, the student should preserve the random seed, graph-generation probability, Python version, and source-code commit. These details allow a reviewer to distinguish a genuine algorithmic change from a change in input or environment.

APPENDIX H – VIVA PREPARATION AND FINAL REVIEW

Likely viva questions and concise answers

Question	Answer
Why is this a graph-colouring problem?	Courses are vertices, shared-student conflicts are edges, and time slots are colours.
Why does C1=1 not violate a constraint?	C1 is the first vertex, so no neighbour is coloured yet.
Why is C3 forced to colour 3 in the k=3 trace?	C3 is adjacent to C1=1 and C2=2.
Why does k=2 fail?	C1, C2, C3 form a triangle, which needs three distinct colours.
What is undone?	The most recent accepted colour is reset to 0 when its continuation fails.
Is every rejection a backtrack?	No. A rejection is a local illegal candidate; a backtrack follows an exhausted recursive call.
What is the requested complexity?	$O(k^n)$ candidate paths, with polynomial conflict-check overhead.
Why not use a heuristic in Module 2?	The plain baseline must remain fixed for comparison with the enhanced module.
Is the timetable operationally complete?	No. Rooms, capacity, invigilators, and preferences are outside this simplified model.

Final academic review questions

- Can I explain every row in the k = 3 and k = 4 tables without reading the answer?
- Can I identify the exact line in the code that performs undo?
- Can I justify why the graph needs at least three colours?
- Can I distinguish attempts, recursive calls, backtracks, and elapsed time?
- Can I reproduce the experiment using the stated seeds?
- Have I replaced all personal-information placeholders and added the GitHub URL?
- Have I checked that my final repository contains the same source version described in the report?

This expanded supplement is intended to convert the report from a result summary into a study document: it records the model, the state transitions, the failure mechanism, the experiment protocol, and the questions likely to arise during evaluation.